

INFO8003-1: UAV Control via Deep Reinforcement Learning

Arthur de Landsheere, Yanis Geurts

May 2026

1 Introduction

Autonomous drone control is a challenging problem at the intersection of control theory and machine learning. Traditional control approaches rely on hand-crafted controllers that require precise system models and struggle to generalize across diverse operating conditions. Deep Reinforcement Learning (DRL) offers an alternative paradigm: rather than explicitly programming a control policy, an agent learns to act through trial-and-error interaction with an environment, optimizing a reward signal that encodes the desired behavior.

In this project, we apply DRL to the control of a quadrotor UAV using the PyFlyt simulator, a Gymnasium-compatible environment built on the PyBullet physics engine. We investigate two distinct tasks of increasing complexity. The first is a stabilization task, where the agent must maintain a fixed hovering position. The second is a navigation task, where the agent must sequentially reach a series of randomly placed 3D waypoints across a large flight dome. Additionally, we participate in a dogfight tournament where a fixed-wing agent must defeat an opponent in aerial combat.

For each task, we train and compare two algorithms with fundamentally different properties: Proximal Policy Optimization (PPO), an on-policy method known for its stability and simplicity, and Soft Actor-Critic (SAC), an off-policy maximum entropy algorithm that encourages exploration through entropy regularization. Both algorithms are evaluated across multiple flight modes, which abstract low-level motor control to varying degrees — from raw motor PWM commands to high-level velocity or position setpoints — allowing us to study how the level of action abstraction affects learning speed and final performance.

A key contribution of our experimental setup is the use of curriculum learning for the waypoint navigation task, where we progressively increase task difficulty across training phases by expanding the flight dome and the number of active waypoints. All experiments are tracked with Weights & Biases, logging episode rewards and deterministic evaluation rewards to ensure reproducibility and support rigorous comparison. The project can be found at <https://wandb.ai/ChelseaCity/RL-Drone-Project>, and the codebase is available at <https://github.com/YanisGrts/Info8003-Project/tree/main>.

2 Hover Environment

The objective of the **QuadX-Hover-v4** environment is to maintain a quadrotor at a fixed target position with a stable orientation. This section outlines the formalization of this task, the algorithms selected, and a comparative analysis of their performance across different flight modes.

2.1 MDP Formalization

To solve this environment, the problem was formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, R, T, γ) . Following is the detailed environment found by inspecting the handed code and the PyFlyt Github repository (<https://github.com/jjshoots/PyFlyt>).

- **State Space (S):** The environment provides a continuous observation space consisting of a 21-dimensional vector ($S \in \mathbb{R}^{21}$). The array is a concatenation of the following vectors:
 - **ang_vel** (3 values): The angular velocity of the drone (roll, pitch, yaw rates).
 - **quaternion** (4 values): The attitude/orientation of the drone in 3D space.
 - **lin_vel** (3 values): The linear velocity (x, y, z speeds).
 - **lin_pos** (3 values): The linear position of the drone in 3D space.
 - **previous_action** (4 values): The action the agent took in the previous timestep.
 - **aux_state** (4 values): Auxiliary information.
- **Action Space (A):** The action space is a 4-dimensional continuous vector. Through inspection of the environment, we found that the action space’s first three values were bounded in $[-\pi; \pi]$, while the fourth value is bounded in $[0; 0.8]$. The semantic meaning of these four continuous values depends entirely on the selected flight mode. For this environment, we are asked to focus on flight mode 0. In this mode, the actions are: $A = (\dot{p}, \dot{q}, \dot{r}, T)$, which represents the angular velocity and the thrust.
- **Reward Function (R):** The reward function is mathematically designed to promote stable hovering at exactly 1 meter above the origin, while penalizing crashes. The exact step reward is formulated as:

$$R = -0.1 + env_step_ratio(1.0 - (D_{linear} + D_{angular} + 0.01 \cdot \omega_z^2)) + R_{terminal}$$

- The -0.1 penalty is a constant step penalty.
- The env_step_ratio is the environment frequency divided by the agent frequency. In this environment, it is equal to $120/40 = 3$.
- the 1.0 term is a survival bonus.
- The $-(D_{linear} + D_{angular} + 0.01 \cdot \omega_z^2)$ measures how far the drone is from a perfect hovering situation. D_{linear} is the distance to the target position, $D_{angular}$ is the deviation from a zero roll-pitch attitude, and $0.01 \cdot \omega_z^2$ penalizes high yaw rates.
- If the drone crashes or gets out of the dome, a penalty of $R_{terminal} = 100$ is applied.

Knowing that, the maximum theoretically achievable cumulative reward is: $R_{MAX} = (-0.1 + 3 \cdot (1 - (0 + 0 + 0)) + 0) \cdot 400 = 1160$, where 400 is the maximum number of steps in a simulation.

- **Transition Dynamics (T):** The transition dynamics are governed by the PyBullet physics engine, which simulates the world at 120 Hz, and the agent interacts with the environment at a control frequency of 40Hz.
- **Termination:** Episodes can terminate early if a collision or out-of-bounds event occurs. Successful episodes are truncated after a maximum duration of 10.0 seconds, corresponding to a maximum horizon of 400 agent timesteps.
- **Discount Factor (γ):** We used $\gamma = 0.99$ as the discount factor for training our algorithms.

2.2 Algorithm Description

We train and compare two deep RL algorithms with fundamentally different properties.

Proximal Policy Optimization (PPO) is an on-policy, actor-critic algorithm that collects a fixed-length rollout buffer using the current policy, then performs several epochs of gradient updates before discarding the data. The core idea is to prevent destructively large policy updates by clipping the probability ratio between the new and old policy within a small interval $[1 - \epsilon, 1 + \epsilon]$, yielding the surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the likelihood ratio and $\hat{A}_t$ is the generalized advantage estimate (GAE). An entropy bonus weighted by β is added to the objective to discourage premature convergence to deterministic policies. PPO is sample-inefficient by design — collected transitions are used only for a few epochs and then thrown away — but it is stable, easy to tune, and parallelizes well across many environment workers. We use State-Dependent Exploration (SDE) in place of the standard diagonal Gaussian policy, which samples a noise vector once per episode reset rather than per step, producing smoother and more temporally consistent actions — a desirable property for continuous drone control.

Soft Actor-Critic (SAC) is an off-policy, actor-critic algorithm built on the maximum entropy RL framework. Instead of maximizing expected return alone, SAC maximizes the objective:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

where $\mathcal{H}(\pi(\cdot|s_t)) = -\log \pi(a_t|s_t)$ is the policy entropy and $\alpha > 0$ is a temperature parameter that controls the trade-off between reward maximization and entropy maximization. High entropy encourages the agent to explore and avoid committing prematurely to suboptimal modes. α is automatically tuned during training by constraining the expected entropy to a target value. SAC maintains a replay buffer of past transitions and updates the policy and two Q-networks (to mitigate overestimation bias) off-policy, making it significantly more sample-efficient than PPO. Target Q-networks are updated via a soft (Polyak) average controlled by τ . As with PPO, we enable SDE for smoother action sequences.

We chose PPO for its robustness and ease of scaling across parallel environments. Its on-policy nature means collected rollouts are always fresh and consistent, which simplifies implementation and avoids the distributional shift issues that can affect off-policy methods in rapidly changing environments. We chose SAC for its sample efficiency and principled exploration through entropy maximization, which is particularly valuable in the early stages of training when the agent must discover useful behaviors from scratch in a large continuous state space. Their complementary strengths make the comparison informative across both the hover stabilization and waypoint navigation tasks.

2.3 Experimental Setup

All experiments were tracked using Weights & Biases to monitor episode rewards and policy videos. Table 1 reports the hyperparameter configuration used for each algorithm on the hover task.

Table 1: Hyperparameter configurations for PPO and SAC on the Hover task.

Hyperparameter	PPO	SAC
Learning rate	1×10^{-4}	3×10^{-4}
Network architecture	[256, 256]	[256, 256]
Batch size	128	256
Horizon / Buffer size	1024 steps	500,000 transitions
Epochs per update	5	—
Entropy coefficient	0.01	auto-tuned
τ (soft update)	—	0.005
Learning starts	—	10,000 steps

2.4 Results

As presented in Table 2, both Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) successfully learned the hovering task under the mandatory flight mode (Mode 0). Both algorithms achieved highly comparable mean rewards, with SAC scoring slightly higher at 1138.33 compared to PPO’s 1137.99. Both results are quite close to the maximum theoretically reachable reward of 1160. However, a notable distinction lies in the stability of the final policies: SAC exhibits a

Table 2: Summary of Mean and Standard Deviation of Rewards by Algorithm and Flight Mode

Algorithm	Flight Mode	Mean Reward	Std. Deviation	Crash Rate
PPO	0	1137.99	1.59	0%
SAC	0	1138.33	0.08	0%

significantly lower standard deviation (0.08) than PPO (1.59). This suggests that the agent trained with SAC converged on a more consistent and robust hovering behavior.

2.5 Flight Mode Comparison

To further evaluate adaptability, we compared the number of environment timesteps required for each algorithm to reach a satisfactory mean episodic reward of 1100. The results are summarized in Table 3.

Table 3: Approximate timesteps required to reach a mean reward of 1100 across different flight modes. "DNR" indicates the algorithm Did Not Reach the threshold.

Flight Mode	SAC Timesteps	PPO Timesteps
Mode -1	DNR	DNR
Mode 0	450,000	1,500,000
Mode 4	350,000	2,000,000
Mode 6	250,000	1,100,000
Mode 7	110,000	1,100,000

The data highlights that SAC is significantly more sample-efficient, requiring fewer environment interactions to converge (e.g., 450,000 steps vs. 1.5 million for PPO in Mode 0). However, it is crucial to note that PPO executes steps significantly faster. SAC’s off-policy architecture requires computationally expensive updates to multiple networks at almost every step, whereas PPO collects data rapidly and performs batched gradient updates.

Mode 7 offered the most intuitive control mapping, yielding the fastest convergence for both algorithms. Except for mode 4 with PPO, the number of required timesteps decreases as the control becomes higher-level. Mode -1 presented an optimization landscape that trapped both agents in a persistent local optimum, failing to reach the 1100 threshold entirely and plateauing at ≈ 950 mean reward.

It must be noted that the PPO agents were trained without observation and reward normalization. Preliminary tests confirmed that standard normalization wrappers drastically reduce the required timesteps. Had normalization been applied, we expect that the sample efficiency gap between SAC and PPO would likely have narrowed.

3 Waypoints Environment

The objective of the **QuadX-Waypoints-v4** environment is to navigate a quadrotor through a sequence of randomly placed 3D target coordinates within a bounded flight dome. Unlike the hover task, which requires maintaining a static position, this environment demands active navigation, longer-horizon planning, and the ability to sequence multiple goals. This section outlines the formalization of this task, the curriculum training pipeline adopted to tackle its difficulty, and a comparative analysis of the performance of PPO and SAC.

3.1 MDP Formalization for Waypoints

The objective of the **QuadX-Waypoints-v4** environment is to navigate a quadrotor through a generated sequence of 3D target coordinates. To solve this environment, the problem was formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, R, T, γ) .

- **State Space (S):** The base environment originally outputs a dictionary space containing the drone’s attitude and a variable-length list of distances to remaining targets. To make this compatible with standard RL algorithms, the `FlattenWaypointEnv` wrapper flattens this into a continuous 33-dimensional vector ($S \in \mathbb{R}^{33}$). This array is a concatenation of the following:
 - **attitude** (21 values): The kinematic state of the drone, composed of angular velocity (3), quaternion orientation (4), linear velocity (3), linear position (3), previous action (4), and auxiliary state variables (4).
 - **target_deltas** (12 values): The relative 3D distance vectors (x, y, z) to a maximum of 4 upcoming waypoints. As waypoints are reached, the remaining unused targets are padded with zeros.
- **Action Space (A):** The action space is a 4-dimensional continuous vector, shared with the Hover environment. The first three values are bounded in $[-\pi, \pi]$, while the fourth value is bounded in $[0, 0.8]$. The semantic meaning of these four continuous values depends entirely on the selected flight mode. For this environment, we are asked to focus on the 6th flight mode. In this mode, the actions are: $A = (\dot{x}, \dot{y}, \dot{r}, \dot{z})$, which represents the ground velocity, yaw rate and vertical velocity.
- **Reward Function (R):** The reward function is engineered to encourage rapid progression toward the next sequential waypoint. If a waypoint is successfully reached, the step reward is overridden by a large flat bonus of 100.0. Otherwise, the step reward is formulated as:

$$R = -0.1 + env_step_ratio(\max(3.0 \cdot \Delta_{progress}, 0) + \frac{0.1}{D_{target}} - 0.01 \cdot \omega_z^2) + R_{terminal}$$

- The -0.1 is a constant timestep penalty to encourage finishing the course quickly.
 - The `env_step_ratio` is the environment frequency divided by the agent frequency. In this environment, it is equal to $120/30 = 4$.
 - The $3.0 \cdot \Delta_{progress}$ term grants a proportional positive reward for moving closer to the target compared to the previous step (clamped at a minimum of 0).
 - The $\frac{0.1}{D_{target}}$ term grants an increasing proximity bonus as the distance to the next target (D_{target}) decreases.
 - The $0.01 \cdot \omega_z^2$ quadratically penalizes high yaw rates (ω_z) to prevent spinning.
 - If the drone crashes or flies out of the bounding dome, a terminal penalty of $R_{terminal} = -100.0$ is applied.
- **Transition Dynamics (T):** The transition dynamics are governed by the PyBullet physics engine, which simulates the world at 120Hz. The agent interacts with the Waypoints environment at a control frequency of 30 Hz.
 - **Termination:** Episodes can terminate early as failures if a collision or out-of-bounds event occurs. A successful episode truncates early if all required waypoints are reached. Otherwise, the environment truncates after a maximum duration of 120.0 seconds, which corresponds to 3600 agent timesteps.
 - **Discount Factor (γ):** We used $\gamma = 0.99$ as the discount factor for training our algorithms.

3.2 Algorithm Description

The same two algorithms described in Section 2.2 — PPO and SAC — are used for the waypoint navigation task. Their core properties (on-policy vs. off-policy, clipped surrogate objective vs. maximum entropy objective, rollout buffer vs. replay buffer) are unchanged. The key differences in this context are the hyperparameter choices, the use of observation normalization via `VecNormalize`, and the curriculum training pipeline detailed in Section 3.3.

Several aspects of the waypoint task make specific algorithm design choices particularly important. First, the reward signal is sparser than in the hover task: the agent receives a large bonus only upon reaching a waypoint, and the dense shaping terms (progress reward, proximity bonus) are relatively weak in early training when the drone rarely approaches a target. This makes exploration critical, and SAC’s entropy maximization objective provides a principled mechanism for it (the agent is explicitly rewarded for visiting diverse states rather than committing prematurely to a suboptimal trajectory). PPO addresses this through the entropy bonus coefficient β , though less adaptively. Second, observation normalization via `VecNormalize` is more important here than in the hover task: the 33-dimensional observation mixes attitude values (typically small, near zero) with target delta vectors whose magnitude scales with the dome size (up to 150m in the final phase), creating a heterogeneous input distribution that can slow or destabilize learning without normalization. Third, State-Dependent Exploration (SDE) is particularly beneficial for navigation: by committing to a consistent noise vector over multiple steps, the agent produces smoother, more directed trajectories toward the target, rather than the erratic step-wise perturbations of standard Gaussian noise that can cause the drone to oscillate around a waypoint without ever triggering the capture radius.

3.3 Experimental Setup

All experiments were tracked using Weights & Biases, logging episode rewards and deterministic evaluation rewards. Training used 8 parallel environment workers for PPO and 16 for SAC, all wrapped with `VecNormalize` for observation normalization (reward normalization was disabled to preserve the shaped reward signal).

Hyperparameters. Table 4 reports the final configuration used for each algorithm on the waypoint task.

Table 4: Hyperparameter configurations for PPO and SAC on the Waypoints task.

Hyperparameter	PPO	SAC
Learning rate	3×10^{-4}	3×10^{-4}
Network architecture	[256, 256]	[256, 256]
Batch size	256	512
Horizon / Buffer size	4096 steps	500,000 transitions
GAE λ	0.95	—
Clip range ϵ	0.2	—
Entropy coefficient β	0.01	auto-tuned
Target entropy	—	-4.0
τ (soft update)	—	0.005
γ (discount)	0.99	0.99
Train frequency	—	every 16 steps
Gradient steps	—	8
Learning starts	—	5,000 steps
State-Dependent Exploration	✓	✓
SDE sample frequency	—	64 steps
$\log \sigma$ init	-2	-3

To arrive at these final configurations, several approaches were investigated. Initial experiments revealed that the waypoint task was significantly harder than the hover task: the agent rarely approached a target by chance, the reward signal remained near-constantly negative, and naive training runs failed to produce any meaningful navigation behaviour. Two strategies were then explored to address this.

First, we attempted automated hyperparameter search using Optuna, implemented in `tune_params.py`. The script ran a study of 30 trials, each training for 50,000 to 100,000 steps and evaluating the mean reward to guide the search. However, this approach proved inconclusive: the hyperparameter configurations selected by Optuna did not transfer to full-length training runs, likely because

50,000 steps is insufficient for the agent to exhibit meaningfully different behaviour across trials, making the evaluation signal too noisy to guide the search reliably.

Reward Shaping. To accelerate learning beyond the base environment reward, a custom `WaypointRewardShaping` wrapper was applied to each environment. At each step, a potential-based shaping term $\phi(s') - \phi(s)$ is added to the reward, where $\phi(s) = -\lambda \|d_0\|$ and d_0 is the vector to the next waypoint, with $\lambda = 0.2$. This provides a dense progress signal without altering the set of optimal policies, following standard potential-based shaping theory. Steps where the base reward indicates a waypoint capture or a crash (i.e. $|r| \geq 10$) are left unshaped to preserve the original terminal signal. Additionally, a fixed time penalty encourages efficiency, and a yaw penalty discourages spinning in place.

3.3.1 Curriculum Learning

Training the agent directly on the full evaluation environment (150 m dome, 4 waypoints) proved unstable in early experiments, as the agent rarely reaches even a single waypoint by chance, yielding a near-constant negative reward signal with no learning gradient. We therefore adopt a curriculum that progressively increases task difficulty, transferring the model checkpoint between phases. Both algorithms follow a curriculum structure, but their specific approaches diverged significantly during experimentation, as detailed below.

PPO Curriculum. PPO was trained on two flight modes: mode 6 (ground velocity control) as the primary navigation mode, and mode 0 (angular velocity and thrust) to evaluate adaptability to a harder control abstraction. For flight mode 0, we adopt a four-phase curriculum as described in Table 5, moving smoothly from a 25 m dome with a single waypoint to the full 150 m evaluation environment. SDE was utilized with a `log_std_init` of -2 and a standard [256, 256] network architecture to handle the 33-dimensional flattened observation space.

Conversely, flight mode 6 proved unexpectedly unstable with standard settings. To stabilize learning, we implemented a custom observation wrapper that filtered the state space down to a 13-dimensional vector, isolating the most critical features: angular velocity, quaternion orientation, linear velocity, and the relative distance to the immediate next target. Furthermore, the curriculum was condensed into two phases: an initial phase with a 20 m dome and 1 waypoint, followed directly by the final 150 m evaluation environment with 4 waypoints. To accommodate this fine-tuning approach, the network capacity was reduced to [128, 128] with a `Tanh` activation function.

Table 5: Curriculum training phases for PPO on the Waypoints task (flight mode 0).

Phase	Dome radius (m)	Waypoints	Timesteps	Goal
1	25	1	3,000,000	Learn basic directed navigation and attitude stabilization toward a single target
2	25	4	3,000,000	Learn to sequence multiple waypoints within a familiar spatial scale
3	75	4	5,000,000	Transfer navigation skills to a larger dome, requiring longer-horizon planning
4	150	4	10,000,000	Master the full evaluation environment

At each phase transition, the model weights are transferred directly from the previous checkpoint. The learning rate is reduced from 3×10^{-4} to 3×10^{-5} and `target_kl` is tightened to 0.015 to prevent destabilization of the already-trained policy when adapting to the new difficulty level.

SAC Curriculum. SAC was trained exclusively on flight mode 6 for the waypoint navigation task. A first curriculum was designed to mirror the structure adopted for PPO, progressing through four phases of increasing dome size and waypoint count (see Table 9 in the Appendix). However, this original curriculum proved unsuccessful: entropy collapse occurred from Phase 2 onward, with the policy converging to a degenerate hovering behaviour rather than active navigation. To address this, three alternative curricula (Runs A, B, and C) were designed and run in parallel, each starting

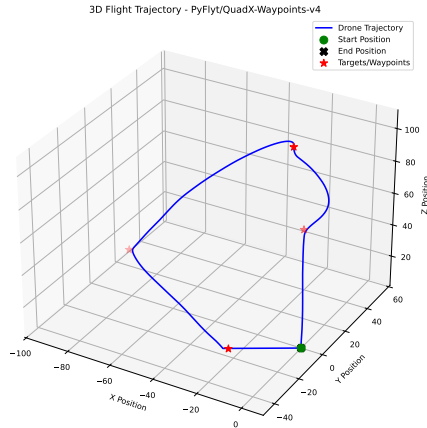


Figure 1: 3D trajectory of our agent trained with PPO on flight mode 0.

from the same Phase 1 checkpoint and transferring weights directly from that checkpoint. Three hyperparameter corrections were applied at each phase transition: `target_entropy` was fixed to -4.0 (correcting the original value of 0.0 which contributed to entropy collapse), the time penalty was reduced from -0.1 to -0.05 per step to prevent the penalty from dominating the reward signal, and the `VecNormalize` running statistics were no longer reset between phases to preserve the normalization built during Phase 1. The three runs explore different pacing strategies: Run A introduces an intermediate 2-waypoint phase before scaling the dome, Run B spends the majority of the budget directly on the final environment, and Run C transfers the Phase 1 model directly to the full Dome 150 m environment for the entire remaining budget. All phase schedules are detailed in Table 9 in the Appendix.

3.4 Results

Table 6: Summary of Mean and Standard Deviation of Rewards, Mean Waypoints, and Number of timesteps, by Algorithm and Flight Mode.

Algo	Flight Mode	Mean Reward	Mean waypoints	Mean Timesteps	Crash Rate
PPO	0	1101.95 ± 364.33	3.85	285.9	5.0%
PPO	6	238.06 ± 253.90	1.20	3070.2	15.0%
SAC	6	-338.27 ± 119.22	0.00	2691.1	40.0%

As detailed in Table 6, the performance of PPO varied drastically depending on the flight mode. In the lower-level flight mode 0, PPO achieved strong performance, reaching a mean reward of 1101.95 and successfully capturing an average of 3.85 out of 4 waypoints. The agent exhibited a low crash rate of 5.0% and completed episodes in an average of 285.9 timesteps. A simulated episode of the obtained agent can be observed in Figure 1.

In contrast, the PPO agent trained on flight mode 6 struggled significantly, achieving a mean reward of only 238.06 and capturing just 1.20 waypoints on average. The agent frequently timed out or failed to navigate between targets, utilizing an average of 3070.2 timesteps per episode and crashing 15.0% of the time. Despite the custom 13-dimensional observation wrapper designed to simplify the state space, the agent failed to generalize to the full 150m environment.

Regarding SAC, the best performing model across all curricula on the final evaluation environment (Dome 150 m, 4 waypoints) was Phase A3 from Run A, which achieved a mean reward of -338.27 ± 119.22 , zero waypoints reached on average, and a crash rate of 40.0%. While this represents the strongest SAC result on the final environment, it remains far below the performance of the two PPO agents reported in Table 6. The full evaluation results across all SAC phases and curricula are reported in Table 10 in the Appendix. A consistent pattern emerges across all curricula: performance degrades sharply as the dome size increases, with Phase 1 models (Dome

25 m, 1 waypoint) showing the most reasonable behaviour and later phases failing to reach any waypoints on average. This confirms that the entropy collapse identified during training was not fully resolved by the hyperparameter corrections applied in Runs A, B, and C.

Comparing PPO and SAC on the waypoint task reveals a stark contrast. PPO under flight mode 0 achieved a mean reward of 1101.95 ± 364.33 and reached an average of 3.85 out of 4 waypoints, while the best SAC model on the final environment reached a mean reward of -338.27 ± 119.22 and failed to capture any waypoints consistently. This reverses the pattern observed in the hover task, where SAC was marginally stronger. The difference is attributable to the entropy collapse issue specific to SAC on this task, combined with the significantly larger training budget allocated to PPO. It would therefore be premature to conclude that PPO is fundamentally better suited to navigation tasks than SAC; the results reflect the specific training difficulties encountered rather than an inherent algorithmic advantage.

A proper statistical evaluation following best practices for deep RL, such as reporting performance across multiple independent seeds and using interval estimates rather than point estimates, was not implemented in this work due to time and computational constraints. The reported statistics, therefore, reflect episode-level variance within individual training runs, and results should be interpreted as indicative rather than as statistically robust conclusions.

3.5 Flight Mode Comparison

A comparative analysis of flight modes yields a highly counterintuitive result: PPO performed vastly better in flight mode 0, despite it being categorized as a more difficult, lower-level control abstraction compared to the high-level ground velocity commands of flight mode 6.

This result, while surprising at first glance, can be attributed to several converging factors. First, flight mode 6 introduces a layer of abstraction via cascaded PID controllers that translate velocity commands into motor outputs. While this simplifies the action semantics, it also introduces a temporal lag between the agent’s command and the drone’s actual response, making credit assignment harder during training. In contrast, flight mode 0 gives the agent more direct control over the drone’s angular velocity and thrust, producing a tighter and more predictable mapping between action and physical outcome. Second, the observation space reduction applied to flight mode 6 (filtering from 33 to 13 dimensions) may have discarded useful information, such as the full attitude state, limiting the agent’s situational awareness. Finally, the curriculum for flight mode 0 comprised four phases and approximately 21 million total training steps, compared to a condensed two-phase curriculum for flight mode 6, meaning the mode 0 agent benefited from significantly more training time and a smoother difficulty progression.

4 DogFight

The final challenge of this project involved an adversarial multi-agent environment:

MAFixedwingDogfightEnvV2. In this 1v1 dogfighting scenario, the agent controls a fixed-wing UAV using a 4-dimensional continuous action space (roll, pitch, yaw, throttle) based on a 37-dimensional observation vector that includes the agent’s kinematic state and the opponent’s relative position.

4.1 Methodology and Reward Shaping

In an adversarial dogfighting environment, relying solely on sparse terminal rewards is highly inefficient. Randomly initialized agents rarely engage one another, resulting in negligible learning signals during early training. To overcome this, we implemented two key strategies: reward shaping and self-play.

The provided custom wrapper `DogfightSelfPlayEnv` is used to convert the multi-agent environment into a single-agent interface. Modifications were made to be able to extract the opponent’s health metric and inject it into dense reward shaping. The agent was rewarded proportionally to the damage dealt to the opponent. The intermediate reward helped bridge the gap between random exploration and the sparse terminal reward obtained when destroying the enemy.

4.2 Training Strategy

For our primary dogfight agent, we employed Proximal Policy Optimization embedded within a self-play curriculum. PPO was selected as the primary algorithm due to its on-policy stability and superior sample efficiency in this specific environment. The self-play curriculum that we have implemented makes the agent train on a historical snapshot of its own policy. This ensures that as the agent improves, the opponent scales in difficulty. The self-play training was conducted across three distinct phases, saving periodic checkpoints to expand the pool of historical opponents. An agent was also trained using Soft Actor Critic to be used as a baseline comparison.

4.3 Results

4.3.1 Evaluation against Heuristic Baselines

To benchmark our trained models before the tournament, we evaluated them against a suite of hand-crafted agents. We selected three heuristic profiles:

- **StraightAgent:** Flies at a constant altitude and heading. Tests basic targeting.
- **EvasiveAgent:** Executes continuous aggressive maneuvers to break missile locks.
- **NoseOnAgent:** A lead-pursuit agent that actively steers to intercept the agent’s predicted position.

Table 7: Win / Loss / Draw rates of trained agents against heuristic baselines (10 matches each).

Agent	vs. Straight	vs. Evasive	vs. Nose-On
PPO (Self-Play)	10 / 0 / 0	10 / 0 / 0	10 / 0 / 0
SAC (Baseline)	10 / 0 / 0	10 / 0 / 0	10 / 0 / 0

Table 7 summarizes the win/loss/draw rates of our trained models against these baselines over 10 evaluation episodes. A match was considered a win if the cumulative reward exceeded +50, and a loss if it fell below −50. Our RL agent achieved a perfect win rate against all three baseline policies. Even the reactive **NoseOnAgent** failed to secure any victories.

4.3.2 Tournament

Finally, we simulated a tournament to establish a robust ranking of our models using the Elo rating system (starting at 1500). The tournament included the three sequential phases of our PPO self-play agent and our SAC agent, with each pair playing 10 matches.

Table 8: Internal Elo rankings of trained agents after a round-robin tournament.

Rank	Model Version	Final Elo
1	PPO Self-Play Phase 1	1528.0
2	PPO Self-Play Phase 3	1509.0
3	PPO Self-Play Phase 2	1487.4
4	SAC Baseline Phase 1	1475.6

As demonstrated in Table 8, the internal tournament yielded a counter-intuitive result: the Phase 1 self-play agent outperformed later phases. This is probably due to a phenomenon known as "policy forgetting". It occurs when an evolving agent over-optimizes for its most recent opponents or tasks, erasing the strategies and generalizations it previously mastered. Because our last agent was optimized to defeat recently developed tactics, it lost its robustness and forgot how to counter the manoeuvrers of Phase 1. Future iterations could mitigate this by training against a randomized pool of all past checkpoints rather than purely sequential ones. Furthermore, SAC ranked last across the board. This was expected as it was trained for a smaller amount of time, and its result would probably have been improved if we had trained it for longer. According to the table, our final submission is the PPO Phase 1 model.

5 Discussion

Several observations have already been discussed in context throughout this report, notably the counterintuitive flight-mode result in Section 3.5 and the sample-efficiency gap between PPO and SAC in the hover-flight-mode comparison in Section 2.5. This section synthesizes those findings and complements them with a broader analysis of algorithm behavior, failure modes, and the limitations imposed by compute constraints.

5.1 Algorithm comparison

5.1.1 Hover environment

In the hovering task, both PPO and SAC successfully solved the environment under multiple flight modes. However, SAC demonstrated superior theoretical sample efficiency, reaching the convergence threshold in fewer environment steps. However, PPO exhibited more stability during training, whereas SAC’s update occasionally caused high-variance oscillations.

5.1.2 Waypoints environment

In the waypoints navigation task, the performance gap between the two algorithms widened in favor of PPO. We have not been able to reach satisfactory results using an agent trained with SAC, with flight mode 0; our agent trained with PPO has been able to reach a satisfactory policy. However, these results do not prove that PPO is fundamentally better than SAC in these tasks.

5.2 Effect of Flight Mode on Waypoints Performance

The effect of flight mode on waypoint navigation performance was already introduced in Section 3.5, where PPO achieved significantly better results under flight mode 0 than under flight mode 6, despite mode 0 being the lower-level, harder control abstraction. This section offers a broader interpretation of that result.

The conventional expectation is that higher-level flight modes, which abstract away low-level motor control via cascaded PID controllers, should make learning easier by reducing the complexity of the action space. This holds true for the hover task, where both PPO and SAC converge faster under higher-level modes (see Table 3). However, the waypoint task introduces a navigational component that interacts differently with the choice of flight mode.

Under flight mode 6, the agent commands ground-frame velocities. While intuitive, this introduces a temporal lag between the issued command and the drone’s actual displacement, because the PID controller must first translate the velocity setpoint into motor commands. This lag makes credit assignment harder: the agent observes the consequence of its action only after several simulation steps, weakening the connection between a decision and its outcome. Under flight mode 0, the agent directly commands angular velocities and thrust, giving it tighter and more immediate control over the drone’s physical state. Although the action semantics are harder to interpret, the mapping from action to physical outcome is more direct and temporally consistent, which may actually facilitate learning in a task where precise directional control matters.

A compounding factor is the difference in training budget between the two configurations. The mode 0 curriculum comprised four phases totalling approximately 21 million steps, while the mode 6 curriculum was condensed into two phases with a fraction of that budget. It is therefore difficult to isolate the effect of flight mode from the effect of training time. With equivalent budgets and identical curricula, the relative advantage of mode 0 over mode 6 may well narrow or even reverse.

5.3 Failure analysis

SAC Entropy Collapse on Waypoints. The most significant failure mode observed in this work was the collapse of the SAC entropy coefficient during waypoint curriculum training. From Phase 2 onward in the original curriculum, the `ent_coef` dropped to near zero within the first hundred thousand steps of each new phase, as visible in the WandB training curves. Once entropy collapsed, the policy became effectively deterministic and stopped exploring, causing episode

lengths to pin at their maximum value and mean reward to stagnate around -300 , a value consistent with the time penalty alone accumulating over a full-length episode with no waypoints reached.

The root cause was a combination of three compounding factors. First, the `target_entropy` was incorrectly set to 0.0 in the original training script. For a 4-dimensional action space, the theoretically motivated default is $-\dim(\mathcal{A}) = -4.0$. Setting the target to 0.0 forces SAC’s automatic entropy tuning to drive α toward zero in order to satisfy an entropy constraint that is impossible to maintain while also maximizing reward; the policy cannot simultaneously be maximally stochastic and make progress toward distant waypoints. Second, the time penalty of -0.1 per step created a reward floor of approximately -340 over a maximum-length episode. In the larger dome phases where waypoints are rarely reached, this penalty dominated the reward signal, making hovering in place a locally rational strategy since it at least avoided the crash penalty. Third, the `VecNormalize` running statistics were reset at each phase transition, forcing the observation normalizer to re-warm-up from scratch and destabilizing the first tens of thousands of steps of each new phase precisely when the policy was most fragile. These three factors were corrected in Runs A, B, and C by setting `target_entropy` to -4.0 , reducing the time penalty to -0.05 , and preserving the normalization statistics across transitions. While these corrections mitigated the collapse, they did not fully resolve it, suggesting that entropy collapse in this setting is a deeper structural issue rather than a consequence of misconfigured hyperparameters alone. It is likely that the reward signal in the larger dome phases remains too sparse to sustain meaningful exploration regardless of the entropy target, and that a fundamentally different exploration strategy or a finer-grained curriculum would be required to overcome it reliably.

Mode -1 Plateau on Hover. Both PPO and SAC failed to reach the 1100 reward threshold under flight mode -1 , plateauing at approximately 950 . Flight mode -1 exposes raw motor PWM commands directly to the agent, removing all PID abstraction. The optimization landscape under this parameterization is qualitatively different from higher modes: small changes in motor commands produce highly nonlinear and coupled effects on the drone’s attitude and position, creating a reward surface with many shallow local optima where the drone achieves partial stabilization but cannot improve further. Both algorithms appear to have converged to one such optimum (a configuration where the drone achieves rough altitude maintenance but cannot simultaneously minimize the angular deviation and yaw rate penalties required to push the reward above 1100). Escaping such optima would likely require either longer training, a more aggressive exploration strategy, or a curriculum that starts from a higher-level mode and progressively reduces the control abstraction.

PPO Reward Plateaus. PPO suffered severe reward plateaus during curriculum transitions, particularly when increasing the waypoint count from 1 to 4 or expanding the dome radius. In the initial phase, PPO rapidly optimized for flying to a single coordinate. When transitioned to the 4-waypoint phase, the agent’s reward frequently plateaued for millions of steps, and the agent’s standard deviation started increasing until it reached π , meaning that the policy degraded into a near-uniform random walk. Overcoming this phenomenon was a challenge that we had not been able to entirely resolve for the 6th flight mode.

Action Distribution Collapse. Another challenge encountered during the waypoint curriculum concerns premature variance collapse in the continuous action space. In the initial training phases, the agent rapidly converged on a confident policy to reach nearby waypoints, causing the standard deviation of its action distribution to shrink considerably. It proved detrimental when changing the environment configuration. The new environment required renewed, broad exploration to discover new behaviour. However, the agent’s action variance prevented it from deviating from its established flight patterns. To overcome this limitation, it was necessary to artificially re-inflate the standard deviation of the action distribution (by resetting the `log_std` parameter in PPO). This forced the agent to resume exploring the expanded state space.

Crash Analysis. Post-simulation analysis of the PPO agents revealed that crashes are primarily caused by the placement of waypoints rather than by unstable flight behavior. Waypoints are

spawned randomly within the dome, and some configurations place targets close to the floor or near the dome boundary. Since the learned policy has no explicit awareness of its proximity to these limits (the drone’s behavior depends solely on the relative vector to the next waypoint and its current kinematic state, not on its absolute position within the dome), it continues to navigate aggressively toward the target without modulating its trajectory to avoid the boundaries. When a waypoint is close to the floor or the dome edge, this blind pursuit can lead the drone to fly into the boundary before reaching the target. This suggests that the crash rate is unlikely to be reduced by further training alone, and that a more robust solution would require either incorporating absolute position information into the observation space, adding a boundary-avoidance penalty to the reward function, or constraining waypoint spawning to a safe inner region of the dome.

Dogfight Policy Forgetting. A final notable failure mode occurred during the multi-agent dogfight training. As established in the tournament results, the Phase 1 PPO agent consistently defeated the Phase 3 PPO agent. This highlights a classic failure of naive self-play known as policy forgetting or cyclic dynamics. Because the agent was only ever trained against the immediately preceding snapshot of itself, the Phase 3 agent over-optimized to counter the specific evasive maneuvers of Phase 2, fundamentally "forgetting" how to execute or counter the aggressive, foundational tactics it learned in Phase 1.

5.4 Compute and Time Limitations

The experiments conducted in this work were constrained by both computational resources and time. The project was conducted in parallel with other coursework throughout the semester, leaving limited time for systematic experimentation, extended training runs, and thorough analysis. This section discusses what would have been done differently given more compute and time.

Multi-seed evaluation. As acknowledged in the results sections, each configuration was trained with a single random seed due to time and computational constraints. With more resources, every algorithm and flight mode combination would have been trained across at least three to five independent seeds, enabling the use of proper statistical tools such as stratified bootstrap confidence intervals. This would have allowed more reliable conclusions about which algorithm genuinely outperforms the other, rather than relying on single-run observations that may not be representative.

SAC curriculum on waypoints. The SAC curriculum for the waypoint task was the most resource-intensive part of this project. The original curriculum required 21 million steps across four phases, and the three corrective runs (A, B, C) added another 15 million steps in total. These runs were designed and launched late in the project timeline, leaving little time to analyze their results and iterate. With more time and computing power, a more systematic search over curriculum pacing strategies would have been possible, for instance, introducing intermediate dome sizes between 25 m and 75 m, or adding a dedicated phase at 2 waypoints before transitioning to 4. More importantly, each corrective run would have been trained for a longer budget on the final Dome 150 m environment, as 2 to 3.5 million steps on the full environment is likely insufficient for SAC to fully overcome the sparse reward signal at that scale.

Hyperparameter tuning. The Optuna-based hyperparameter search implemented in `tune_params.py` proved inconclusive because each trial was limited to 50,000 steps — too few for meaningful behavioral differences to emerge. This budget was chosen as a compromise between search quality and wall-clock time, given the project deadline. With more time available, each trial could have been run for at least 500,000 steps, producing a more reliable evaluation signal and increasing the likelihood that the best configurations found during search would transfer to full training runs.

Flight mode exploration. Due to time constraints, SAC was only trained on flight mode 6 for the waypoint task, and flight mode -1 was not fully explored for either algorithm on hover. With

more resources, a full grid of algorithms and flight mode combinations would have been trained to completion, enabling a more rigorous and complete flight mode comparison. In particular, a curriculum approach for flight mode -1 (e.g., starting from a higher-level mode and progressively reducing the control abstraction) would have been an interesting avenue to test, given that both algorithms appeared to converge to a local optimum under that mode.

6 Conclusion

This project applied Deep Reinforcement learning to UAV tasks, ranging from basic stabilization to complex 3D navigation and adversarial fighting. Through extensive experimentation with PyFlyt, we demonstrated that on-policy methods like PPO provide the stability required to survive curriculum transition and multi-agent self-play.

We discovered that training an agent in a large-scale, sparse reward environment is inefficient, making structured curriculum learning and dense reward shaping practical tools for advanced control tasks. Additionally, our analysis of flight modes revealed that higher-level control could unexpectedly harm performance in waypoint environments. Finally, our dogfighting results showed the usefulness of self-play in synthesizing adversarial tactics, as well as the importance of cyclic policy regression.

Ultimately, this project showed us the potential of Deep Reinforcement Learning while also allowing us to encounter classical Reinforcement learning challenges.

A Detailed Curricula for SAC on the Waypoints environment

This appendix reports the detailed phase schedules and evaluation results for all SAC curriculum training runs conducted for the waypoint navigation task. The original curriculum served as the baseline; Runs A, B, and C were designed as corrected alternatives after the original curriculum exhibited entropy collapse from Phase 2 onward. Table 9 details the phase schedules for each curriculum, and Table 10 reports the corresponding evaluation results.

Table 9: SAC curriculum training schedules. All curricula share the same Phase 1 starting point (Dome 25 m, 1 waypoint, 3M steps, trained from scratch). Curricula A, B, and C additionally apply three fixes absent from the original: `target_entropy` corrected to -4.0 , time penalty reduced from -0.1 to -0.05 , and `VecNormalize` running statistics no longer reset at phase transitions.

Curriculum	Phase	Dome (m)	Waypoints	Steps	Goal
Original	1	25	1	3M	Basic navigation, single target
	2	25	4	3M	Waypoint sequencing, familiar space
	3	75	4	5M	Transfer to larger dome
	4	150	4	10M	Master full evaluation environment
Run A	A1	25	2	1.5M	Consolidate multi-waypoint nav in small dome
	A2	75	4	1.5M	Scale dome and waypoints together
	A3	150	4	2M	Master full evaluation environment
Run B	B1	25	4	1.5M	Force 4-waypoint learning in small dome
	B2	150	4	3.5M	Spend full budget on final environment
Run C	C1	150	4	5M	Direct transfer from Phase 1, full budget on final env

Table 10: Evaluation results for all SAC curriculum phases on the waypoint task (flight mode 6). Each phase is evaluated on its own training dome size and waypoint count. † indicates models that were not evaluated as training showed no sign of improvement toward the task objective.

Phase	Mean Reward	Mean Waypoints	Mean Timesteps	Crash Rate
1	66.11 ± 113.48	0.90/1	436.1	10.0%
2	-126.70 ± 75.07	1.20/4	2860.5	50.0%
3	-346.68 ± 112.09	0.00/4	3351.7	10.0%
4 (†)	—	—	—	—
A1	100.44 ± 226.47	1.70/2	936.7	0.0%
A2	-380.16 ± 166.40	0.10/4	3085.2	20.0%
A3	-338.27 ± 119.22	0.00/4	2691.1	40.0%
B1	-25.91 ± 216.02	1.60/4	2612.1	20.0%
B2	-421.25 ± 168.65	0.00/4	2984.8	30.0%
C1 (†)	—	—	—	—